# Preuve d'un algorithme

Il est nécessaire de pouvoir s'assurer qu'un algorithme effectue bien ce qui est attendu, qu'il finit bien par s'arrêter et qu'il effectue son travail dans un délai raisonnable : c'est la **preuve** d'un algorithme.

**Définition :**
Réaliser la **preuve** d'un algorithme, c'est :
Prouver qu'il se termine : on parle de **terminaison.**
Prouver qu'il fait bien ce que l'on attend de lui : on parle de **correction.**

## I) TERMINAISON D'UN ALGORITHME :

L'algorithme doit se terminer à un moment. Les séquences d'instruction se composent notamment de boucles et de tests. Il est donc nécessaire de s'assurer qu'aucune erreur de conception ou d'implémentation n'empêche l'algorithme de se terminer.

Dans le cas des boucles, on distingue les boucles de type for et celles de type while.

### 1°) Les boucles de type for :

Avec ce type de boucle, le nombre de tours de boucle, appelés itérations, est explicitement défini. La sortie de boucle est donc certaine, à l'issue du parcours d'un nombre initialement fixé de valeurs.
Par exemple :

```
nombres = [3,9,12,17,41]
for nombre in nombres :
    print(nombre)
```

On obtient :

```
3
9
12
17
41
```

Pour faire émerger plus explicitement la notion de variant de boucle, nous réécrivons le code qui précède avec la notation indicielle, en remplacement de la notation *for… in* typique de Python.

```
for index in range(len(nombres)) :
    print(nombres[index])
```

On obtient le même affichage que précédemment. La variable index va prendre successivement toutes les valeurs générées par la séquence range à partie de la longueur de la liste.

Pour être encore plus explicite, on peut demander d'afficher la valeur de la variable index correspondant à la position de l'élément dans la liste.

```
for index in range(len(nombres)) :
    print(index,nombres[index])
```

On obtient l'affichage suivant :

```
0       3
1       9
2       12
3       17
4       41
```

La variable index prend bien les valeurs successives correspondant à chaque élément de la lise considérée. Notre algorithme comporte un variant de boucle.

**Définition :**
Un variant de boucle est une expression dont la valeur varie à chaque itération.
Il s'agit d'une quantité entière qui :

- doit être positive ou nulle pour rester dans la boucle ;
- doit décroître à chaque itération.

Si l'on arrive trouver une telle quantité, il est évident que l'on va nécessairement sortir de la boucle au bout d'un nombre fini d'itérations, puisqu'un entier positif ne peut décroître infiniment.

2°) Les boucles de type while :

Avec ce type de boucle, le nombre d'itérations peut être indéterminé ; il existe donc un risque que l'algorithme ne se termine jamais s'il est mal conçu ou mal implémenté.
Le programme suivant affiche les nombres de 1 à 10 :

```
i = 0
while i != 11 :
    print(i)
    i += 1
```

**Attention :**
Si on omet la première ligne qui incrémente la valeur de la variable i, elle conservera indéfiniment sa valeur initiale. La condition testée au niveau de l'instruction while restera toujours vraie. Le programme réalise alors une infinité de boucles, souvent désignée boucle infinie, qui nécessite un arrêt forcé pour reprendre la main.

On peut facilement créer une boucle infinie avec un test incorrect par apport au variant de boucle. Si nous décidons de modifier notre programme pour n'afficher que les nombres pairs en modifiant la valeur d'incrémentation, nous créons aussi une boucle infinie.

```
i = 0
while i != 11 :
    print(i)
    i += 2
    # Attention, ce programme crée une boucle infinie !!
    # ctrl+c pour l'arrêter !!
```

Notre programme ne calcule que des nombres pairs, et la condition évalue au niveau de l'instruction while une non égalité avec le nombre impair 11. Cette condition ne sera donc jamais fausse et le programme ne se terminera pas. On corrigera aisément ce code en indiquant une valeur paire ou en remplaçant la non égalité par une inégalité d'infériorité.

Quand les expressions évaluées par l'instruction while sont simples, il est assez facile de se prémunir contre une boucle infinie. Avec une expression plus complexe, la condition de sortie de boucle peut très bien ne pas se réaliser dans certains cas. Il fut donc prendre soin de toujours définir des conditions de sortie qui doivent finir par se réaliser quelles que soient les données fournies à l'algorithme.
Si la boucle n'a pas d'élément variant comme dans notre premier exemple (omission de l'incrémentation) ou que les valeurs prises par l'élément variant ne correspondent pas à l'évaluation e l'expression conditionnelle à l'origine de la boucle, la terminaison ne peut pas se produire.

Un algorithme doit se terminer, mais la terminaison d'un algorithme ne garantit pas la correction du résultat qu'il fournit.

## II) CORRECTION D'UN ALGORITHME :

La démarche pour prouver la correction d'un algorithme consiste à trouver un invariant de boucle.

**Définition :**
Un invariant de boucle est une propriété qui est vraie avant l'exécution d'une itération, et qui reste vraie après cette exécution. L'invariant de boucle n'est pas altéré par cette itération.

Découvrons cette notion avec un algorithme qui recherche et retourne la valeur minimale parmi une liste de valeurs.

```
def minimale(liste) :
    mini = liste[0]
    for element in liste :
        if element < mini :
            mini = element
    return mini
```

Nous testons cet algorithme avec une liste non ordonnée de valeurs.

```
nombres = [-15,2,9,137,-444,12,22]
print(minimale(nombres)
```

On obtient :

```
-444
```

Nous constatons qu'il se termine et il semble fonctionner correctement. Recherchons une propriété qui reste vraie avant et après chaque itération.
L'algorithme procède élément par élément et ajuste la valeur de sa variable locale mini en fonction des valeurs rencontrées. Si une valeur est inférieure, elle devient le nouveau mini. On peut donc considérer qu'à chaque instant la variable mini est toujours inférieure ou égale aux valeurs déjà rencontrées.

**<u>Vérifions-le, étape par étape</u> :**
**Avant l'entrée dans la boucle (INITIALISATION)**, la variable mini est initialisée avec la première valeur de la liste. Elle est donc inférieure ou égale à cette valeur.
**Pour chaque tour de boucle (CONSERVATION)** la condition est vraie à l'entrée dans la boucle. Elle l'est aussi à la sortie de chaque tour de boucle, puisque sa valeur est chaque fois comparée à un nouvel élément de la liste :

- Si l'élément courant est plus grand, la variable mini reste inchangée et est donc inférieure ou égale à toutes les valeurs déjà rencontrées ;
- Si l'élément courant est plus petit, la variable mini prend sa valeur pour devenir inférieure ou égale à toutes les valeurs déjà rencontrées en sortie de boucle.

**Lorsque l'ensemble des itérations est terminé (TERMINAISON)**, la liste a été parcourue en totalité et la variable mini est bien inférieure ou égale à toutes les valeurs de la liste.

La propriété choisie est bien un invariant de boucle. Notre algorithme est donc correct.

<u>Remarque</u> :
Le fait que la valeur de la variable mini puisse varier au cours de l'exécution n'est pas un problème, car notre invariant de boucle n'est pas sa valeur mais une propriété qui lui est rattachée : le fait qu'elle soit à tout moment inférieure ou égale aux valeurs déjà rencontrées.

S'il est relativement aisé de trouver un invariant de boucle pour des algorithmes simples, la tâche peut s'avérer plus délicate avec des algorithmes élaborés.

**A retenir :**

- **Terminaison :** Il s'agit de vérifier ici que les calculs effectués par l'algorithme s'arrêtent bien. Notamment, lorsqu'une boucle conditionnelle est effectuée par l'algorithme, il est primordial de vérifier que l'on sort bien de cette boucle. On parle aussi de **variants de boucle**. L'existence d'un variant de boucle est nécessaire à la terminaison d'un algorithme.
- **Correction :** Il s'agît ici de prouver que l'algorithme fait bien ce qu'on lui demande. Pour cela, on va chercher **un invariant de boucle.** C'est une propriété qui est vérifiée avant l'entrée dans la boucle, qui est vérifiée à chaque itération de la boucle et qui amène au résultat escompté à la sortie de la boucle. L'existence d'un invariant de boucle est nécessaire à la correction d'un algorithme.

## III) <u>APPLICATIONS</u> :

1°) <u>L'âge de Toutou</u> :

Un être humain et un chien ne vieillissent pas de la même manière. Pour déterminer l'équivalent humain de l'âge d'un chien de moins de 15 kg, on utilise le tableau ci-dessous, où x représente l'âge réel de l'animal (en années), et y l'équivalent humain en terme de vieillissement.

| x | y |
|---|---|
| moins de 2 | 8x + 12 |
| de 2 à 17 | 4x + 20 |
| de 17 à 19 | 6x – 20 |
| 19 et plus | 10x – 90 |

Voici l'algorithme correspondant :

```
Fonction AgeHumain(x)
début
   si x >= 19 alors
      Donner à y la valeur 10*x - 90
   sinon
      si x >= 17 alors
         Donner à y la valeur 6*x - 14
      sinon
         si x>= 2 alors
            Donner à y la valeur 4*x + 20
         sinon
            Donner à y la valeur 8*x + 12
         fin
      fin
   fin
   retourner : y
fin
```

Réaliser la preuve de cet algorithme, en étudiant sa terminaison et sa correction.

2°) Somme des n premiers entiers naturels :

Voici l'algorithme :

```
Fonction Somme(n)
début
   Donner à S la valeur 0
   pour i de 1 à n faire
      Donner à S la valeur S + i
  fin
retourner : S
fin
```

Réaliser la preuve de cet algorithme, en étudiant sa terminaison et sa correction.

3°) Retour sur les tris :

**a) Le tri par insertion :**

A priori, les algorithmes de tri par insertion et de tri par sélection "fonctionnent" correctement : ils trient bien le tableau donné en entrée, on dit que ces algorithmes sont corrects. On parle aussi de la "correction d'un algorithme" pour dire qu'un algorithme produit bien le résultat attendu.

Multiplier les exemples qui "fonctionnent" ne veut pas dire que l'algorithme donnera le "bon résultat" dans toutes les circonstances. C'est un peu comme en mathématiques, vérifier qu'une propriété est vraie sur un exemple n'a pas valeur de démonstration. Il se pourrait très bien que dans une situation donnée notre algorithme ne donne pas le résultat attendu.

Il existe un moyen de démontrer (au sens mathématique du terme) la correction d'un algorithme. Si nous arrivons à démontrer la correction de l'algorithme de tri par insertion, nous pourrons alors affirmer que, quel que soit le tableau donné en entrée, nous obtiendrons bien en sortie ce même tableau, mais trié. Pour démontrer la correction de l'algorithme de tri par insertion, nous allons utiliser un "invariant de boucle"

Qu'est-ce qu'un invariant de boucle ?

**On appelle invariant de boucle une propriété qui est vraie avant et après chaque itération.**

Prenons tout de suite un exemple avec le tri par insertion :

```
VARIABLE
t : tableau d'entiers
i : nombre entier
j : nombre entier
k : nombre entier
DEBUT
j←2
tant que j<=longueur(t):   //boucle 1
  i←j-1
  k←t[j]
  tant que i>0 et que t[i]>k:   //boucle 2
    t[i+1]←t[i]
    i←i-1
  fin tant que
  t[i+1]←k
  j←j+1
fin tant que
FIN
```

**Preuve de cet algorithme :**

Divisons le tableau t en 2 parties :

- Une partie qui contient les éléments d'index 1 à j-1 (le j correspond à la variable j dans l'algorithme ci-dessus). Nous noterons cette partie (ce tableau) t[1..j-1]
- Une seconde partie qui contient le reste du tableau t (index j à n si le tableau t comporte n élément)

Nous allons travailler sur cet exemple : t = [27, 10, 12, 8, 11]

Au départ nous avons j=2, donc t[1..j-1] = t[1..1] = [27] (le tableau t[1..j-1] contient un unique élément)

À début de la deuxième itération de la "boucle 1", nous avons t = [10, 27, 12, 8, 11] et j=3, d'où t[1..j-1] = t[1..2] = [10, 27]

À début de la troisième itération de la "boucle 1", nous avons t = [10, 12, 27, 8, 11] et j=4, d'où t[1..j-1] = t[1..3] = [10, 12, 27]

À début de la quatrième itération de la "boucle 1", nous avons t = [8, 10, 12, 27, 11] et j=5, d'où t[1..j-1] = t[1..4] = [8, 10, 12, 27]

a) 1) Que peut-on dire du tableau t[1..j-1] au début de chaque itération ?

Trouver ce qui pourrait être un invariant de boucle est une chose, encore faut-il ensuite démontrer qu'il est correct (une fois de plus l'étude d'un cas particulier ne vaut pas démonstration). La démonstration doit se faire en 3 étapes :

- INITIALISATION : on doit montrer que l'invariant de boucle est vrai avant la première itération de la boucle
- CONSERVATION : on doit montrer que si l'invariant de boucle est vrai avant une itération de la boucle, il le reste avant l'itération suivante.
- TERMINAISON : une fois la boucle terminée, l'invariant fournit une propriété utile qui aide à montrer la correction de l'algorithme.

Revenons au tri par insertion et à notre invariant de boucle :

- INITIALISATION : avant la 1re itération de la boucle 1, le tableau t[1..j-1] contient un seul élément (j=2). Un tableau contenant un seul élément est forcément trié (trivial), la propriété "le tableau t[1..j-1] est trié" est donc vrai.

- CONSERVATION : nous devons maintenant montrer que l'invariant est conservé au cours d'une itération : si l'invariant est vrai en début d'itération (début de la boucle 1), il reste vrai à la fin de cette même itération. Globalement, que fait la boucle 1 ? Elle permet de déplacer les éléments t[j-1], t[j-2], t[j-3], etc d'une position vers la droite jusqu'à ce que l'on trouve la bonne position pour t[j]. Après ces décalages vers la droite effectués, on insère la valeur t[j] au bon endroit. Au cours d'une itération de la boucle 1, nous passons d'un tableau t[1..j] non trié (le tableau t[1..j-1] est trié, mais l'élément t[j] n'est pas encore à la bonne place, donc t[1..j] n'est pas encore trié), à un tableau trié (décaler vers la droite les éléments t[j-1], t[j-2], t[j-3], etc et placer t[j] à la bonne position). En fin d'itération (juste avant le "j←j+1") t[1..j] est trié, juste après le "j←j+1" le tableau t[1..j] précédemment évoqué devient donc le tableau t[1..j-1]. Conclusion : en toute fin d'itération

(après le "j←j+1") le tableau t[1..j-1] est trié. Au début d'une itération t[1..j-1] est trié, à la toute fin de cette même itération (après le "j←j+1") le tableau t[1..j-1] est trié, nous pouvons donc affirmer que si "le tableau t[1..j-1] est trié" est vrai avant une itération de la boucle 1, il le reste avant l'itération suivante.

- TERMINAISON : Que se passe-t-il quand la boucle se termine ? La boucle 1 se termine quand j > longueur(t) (quand j > n pour un tableau comportant n éléments). Quand la boucle se termine, on doit avoir j = n + 1. Si on substitue n+1 par j dans l'invariant de boucle, on obtient le tableau t[1..n]. Quand la boucle se termine, l'affirmation "le tableau t[1..j-1] est trié" devient "le tableau t[1..n] est trié". Or, le tableau t[1..n] correspond au tableau qui est composé des n éléments. Le tableau d'origine est donc maintenant trié.

Cette démonstration nous permet d'affirmer que l'algorithme de tri par insertion est correct.

a) 2) Dans cette démonstration, indiquer quel est l'invariant de boucle.

**b) <u>Le tri par sélection</u> :**
**Tri par sélection du minimum.**
Soit L=[45, 2, 4, 6, -5, 4, 3]
On donne les différents états de la liste L lors du déroulement du tri par sélection.
L=[45, 2, 4, 6, -5, 4, 3]
L=[ -5, 45, 2, 4, 6, 4, 3 ]
L=[ -5, 2, 45, 4, 6, 4, 3 ]
L=[ -5, 2, 3 , 45, 4, 6, 4 ]
L=[ -5, 2, 3, 4 , 45, 4, 6 ]
L=[ -5, 2, 3, 4, 4 , 45, 6 ]
L=[ -5, 2, 3, 4, 4, 6 , 45 ]
L=[ -5, 2, 3, 4, 4, 6, 45 ]

Traitement :

1. On colore en noir tous les éléments.
2. On repère un élément de valeur maximale parmi les éléments noirs.
3. On passe cet élément en queue des éléments noirs puis on le colore en rouge.
4. On reprend au point 2 tant qu'il reste des noirs.

Algorithme :

```
Procédure tri_selection(liste L)
    n ← longueur de la liste L
    i ← 0
    Tant que i < n :
        j ← i
        Tant que j < n :
            Si L[i] > L[j] Alors  v ← L[i]
                                  L[i] ← L[j]
                                  L[j] ← v
            j ← j + 1
        i ← i + 1
        Fin Tant que
    Fin Tant que
```

Réaliser la preuve de cet algorithme, en étudiant sa terminaison et sa correction.